GERMAN UNIVERSITY IN CAIRO (GUC) • FACULTY OF MEDIA ENGINEERING & TECHNOLOGY

DMET902 — Advanced Video Processing / CSEN1121

CINN Project — Winter Semester 2026

Instructors: Prof. Dr. Mohammed Abdel-Megeed Salem, Eng. Yomna Islam Youssef Alayary

Project Title:	**Image Colorization Using From-Scratch Neural Networks & Genetic Algorithm**
Milestone Scope:	**Task 1 (Dataset & Splitting) & Task 2 (Neural Network Architecture)**
Task 2 Deadline:	Sunday, November 15, 2026
Team Name:	`404 Intelligence Not Found` (Archive: `CINN_WS2026_Project_404_Intelligence_Not_Found`)

Team Members

#	Student Name	Student ID	Tutorial	University Email
1	**Mohamed Ayman**	58-8189	T4	`mohamed.awad@student.guc.edu.eg`
2	**Hisham Hani**	58-7992	T3	`hisham.gomaa@student.guc.edu.eg`
3	**Abdallah Amr**	58-2000	T3	`abdallah.bakr@student.guc.edu.eg`
4	**Mohamed Ibrahim**	58-16326	T4	`mohamed.ahmedismail@student.guc.edu.eg`
5	**Ahmed Ashraf**	58-1808	T1	`ahmed.abdelbaky@student.guc.edu.eg`

Part I: Task 1 — Dataset, Preprocessing, and Splitting

1. Executive Summary & Objective

The objective of the course project is to build an end-to-end image colorization engine implemented entirely from scratch without high-level deep learning frameworks (e.g. PyTorch, Keras, TensorFlow). The system learns to map single-channel grayscale images to plausible 3-channel RGB color representations, first through analytical backpropagation (Task 3) and subsequently through evolutionary weight optimization via a Genetic Algorithm (Task 5).

2. Dataset Selection & Domain Rationale

Selected Dataset: CIFAR-10 (Canadian Institute for Advanced Research, 10 Classes)
Target Subdomain: *Animals Domain* — Bird (`2`), Cat (`3`), Deer (`4`), Dog (`5`), Frog (`6`), Horse (`7`)
Curated Sample Count: N = 2,000 images

Theoretical Justification: Standard image colorization trained with Mean Squared Error (MSE) suffers from the regression-to-the-mean phenomenon: if the model is fed radically dissimilar visual categories (e.g., airplanes in blue sky mixed with green frogs and asphalt cars), the loss-minimizing prediction is a desaturated gray-brown average across all pixels. By focusing on a coherent semantic domain (Animals in natural environments), the distribution of plausible colors is tightly bounded to natural earth tones, vegetation greens, sky blues, and warm animal coats. Furthermore, 2,000 samples provides sufficient statistical variety while allowing from-scratch vector operations, gradient propagation, and population-based Genetic Algorithm fitness iterations (Task 5) to execute within reasonable runtimes.

3. Spatial Downsampling & Dimensionality Analysis

In accordance with Task 1 Requirement 1 (*"Keep images small (e.g., 16×16 or 32×32) or work on small patches — a from-scratch network trains slowly"*), images were downsampled from 32×32 to **16×16** using 2×2 spatial block average pooling:

$$Downsampled(i, j, c) = \frac{1}{4} \sum_{u=0}^{1} \sum_{v=0}^{1} Original(2i + u, 2j + v, c)$$

- **Feature Dimensionality at 32×32:** Input = 1,024 grayscale features; Output = 3,072 RGB features. A single hidden layer of 128 neurons requires (1,024 × 128) + (128 × 3,072) ≈ **524,288 weights**.
- **Feature Dimensionality at 16×16:** Input = 256 grayscale features; Output = 768 RGB features. Two hidden layers of [128, 64] require **91,072 parameters** — an **8× reduction in computational complexity**.
- **Critical for Task 5 (Genetic Algorithm):** Chromosomes representing 500k+ real-valued parameters across a population of 50 individuals suffer from severe curse of dimensionality and convergence stagnation. At 16×16 (~91k weights), the genetic algorithm evaluates fitness and converges efficiently.

4. Preprocessing & Feature Engineering

1. **Pixel Intensity Normalization:** Raw integer pixel values in [0, 255] are scaled to floating-point numbers in [0.0, 1.0] via `I_norm = I_raw / 255.0`. This guarantees numerical stability during matrix multiplications and prevents early activation saturation.
2. **Grayscale Input Generation (ITU-R BT.601):** To produce physiologically accurate luminance inputs from normalized RGB targets, we apply the standard ITU-R BT.601 perceptual formula: $Y = 0.299 \times R + 0.587 \times G + 0.114 \times B$, accounting for human trichromatic sensitivity (greatest sensitivity to green, moderate to red, lowest to blue).
3. **Matrix Representations:** Input Matrix: $X \in \mathbb{R}^{2000 \times 256}$; Target Matrix: $Y \in \mathbb{R}^{2000 \times 768}$. Unflattened arrays are preserved for visual rendering.

5. Sanity Checks & Statistical Verification

Channel / Feature	Mean	Std Deviation	Minimum	Maximum
Grayscale Input (Y)	**0.4643**	**0.2169**	0.0000	1.0000
Red Channel (R)	0.4877	0.2274	0.0000	1.0000
Green Channel (G)	0.4656	0.2204	0.0000	1.0000
Blue Channel (B)	0.3959	0.2288	0.0000	1.0000

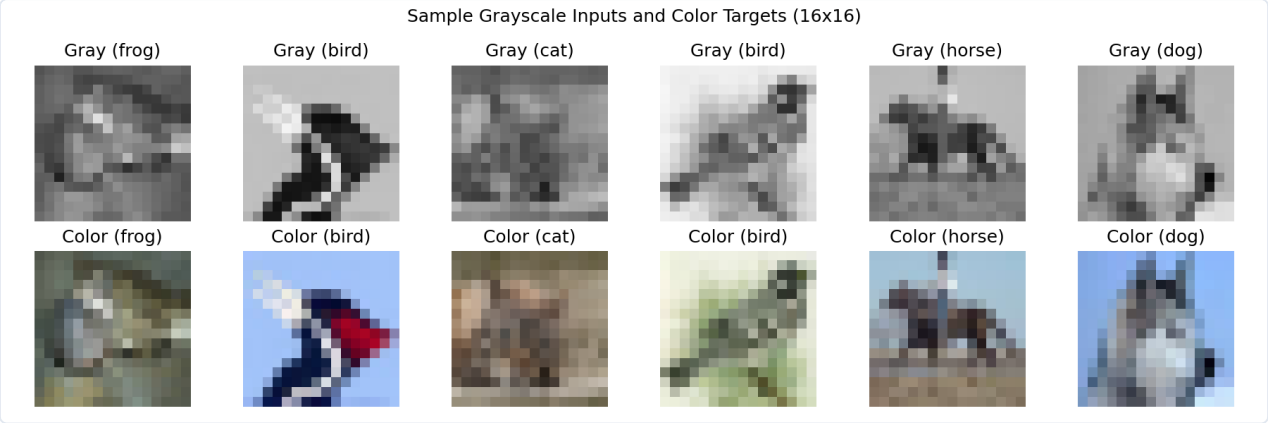


Figure 1: Task 1 Sanity Check — Sample Grayscale Inputs vs. Ground Truth Color Targets (16x16 pixels).

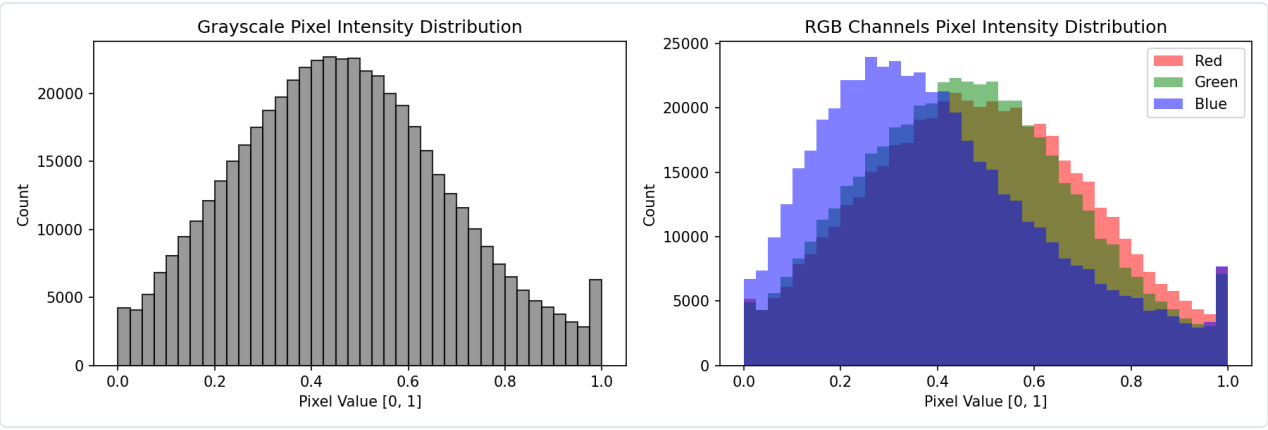


Figure 2: Task 1 Pixel Value Distributions — Grayscale luminance (left) and RGB channel histograms (right).

6. Dataset Splitting & Mini-Batch Partitioning

- **Training Set (70%):** 1,400 samples → `X_train: (1400, 256)`, `Y_train: (1400, 768)`
- **Validation Set (20%):** 400 samples → `X_val: (400, 256)`, `Y_val: (400, 768)`
- **Testing Set (10%):** 200 samples → `X_test: (200, 256)`, `Y_test: (200, 768)`
- **Batching:** The 1,400 training samples are partitioned into **14 mini-batches of exactly 100 samples each**: $X_{train_batches} \in \mathbb{R}^{14 \times 100 \times 256}$ and $Y_{train_batches} \in \mathbb{R}^{14 \times 100 \times 768}$.

Part II: Task 2 — Neural Network Architecture (Built From Scratch)

7. Neural Network Architecture Design

In accordance with Task 2 specifications:

1. **Implementation Paradigm:** Pure object-oriented Python class (`ColorizationNN`) using native NumPy array/matrix operations (`np.dot`). Pre-built layer or model classes (`torch.nn` , `keras.layers` , `sklearn`) are strictly prohibited and avoided.
2. **Input Layer Size:** `n_in = 256` features, corresponding to flattened 16×16 grayscale pixel intensities.
3. **Output Layer Size:** `n_out = 768` features, corresponding to flattened 16×16×3 RGB color channel values.
4. **Hidden Layer Topology:** A deep two-hidden-layer MLP architecture [128, 64] was chosen to provide non-linear capacity for learning luminance-to-chrominance mapping while maintaining parameter tractability.

Layer Dimension & Parameter Breakdown:

Layer	Input Dim	Output Dim	Weight Matrix (W)	Bias Vector (b)	Activation	Parameters
Hidden Layer 1	256	128	(256, 128)	(1, 128)	**ReLU**	32,896
Hidden Layer 2	128	64	(128, 64)	(1, 64)	**ReLU**	8,256
Output Layer	64	768	(64, 768)	(1, 768)	**Sigmoid**	49,920
Total	—	—	—	—	—	**91,072 parameters**

8. Activation Functions Implemented From Scratch

Hidden Layers (ReLU): `f(z) = max(0, z)` . Implemented manually using `np.maximum(0, z)` . ReLU prevents gradient attenuation across layers, computes with extreme efficiency, and enables the network to isolate complex non-linear luminance features.

Output Layer (Sigmoid): `g(z) = 1 / (1 + exp(-clip(z, -500, 500)))` . As mandated by Task 2 Requirement 4 (*"matched to your normalized color range"*), target color pixels reside strictly in [0.0, 1.0]. The Sigmoid function naturally bounds all output predictions to (0, 1), guaranteeing that all generated RGB values are physically valid without artificial post-hoc clamping.

9. Weight and Bias Initialization Strategy

Weights and biases are initialized from scratch without external initializers:

- **ReLU Hidden Layers:** Initialized using He/Kaiming normal scaling: `W ~ N(0, sqrt(2 / n_in))` . This preserves activation variance across layers, avoiding premature vanishing or exploding gradients.
- **Sigmoid Output Layer:** Initialized using Xavier/Glorot normal scaling: `W ~ N(0, sqrt(1 / n_in))` , tailored for logistic units.
- **Biases:** Initialized to zero vectors: `b = np.zeros((1, n_out))` .

10. Forward Pass Propagation & Caching Mechanism

The forward pass is evaluated using vectorized NumPy operations: `Z1 = X • W1 + b1, A1 = ReLU(Z1)` → `Z2 = A1 • W2 + b2, A2 = ReLU(Z2)` → `Z3 = A2 • W3 + b3, Y_hat = Sigmoid(Z3)` . To prepare directly for **Task 3 (Backpropagation)**, all intermediate pre-activations (Z) and activations (A) are cached in a dictionary during execution.

11. Genetic Algorithm Chromosome Compatibility (Task 5 Foresight)

To satisfy Task 5 (*"represent each individual as a full set of weights"*), `ColorizationNN` provides:

- `get_params_vector()` : Serializes all 91,072 weights and biases into a continuous 1D chromosome.
- `set_params_vector(vector)` : Restores 2D weight and bias matrices from a 1D chromosome for fitness evaluation.

12. Verification & Experimental Sanity Results

The network was evaluated on batch 0 of the training set (`X_train_batches[0]`, 100 samples):

- **Input Shape:** (100, 256) → **Output Shape:** (100, 768), perfectly matching (100, 16×16×3).
- **Dynamic Range:** Minimum predicted pixel intensity = 0.0520, Maximum = 0.9407 (strictly in [0, 1]).
- **Initial Un-Trained MSE Loss: 0.076719**, matching theoretical expectations for random Sigmoid activations against normalized pixel values.

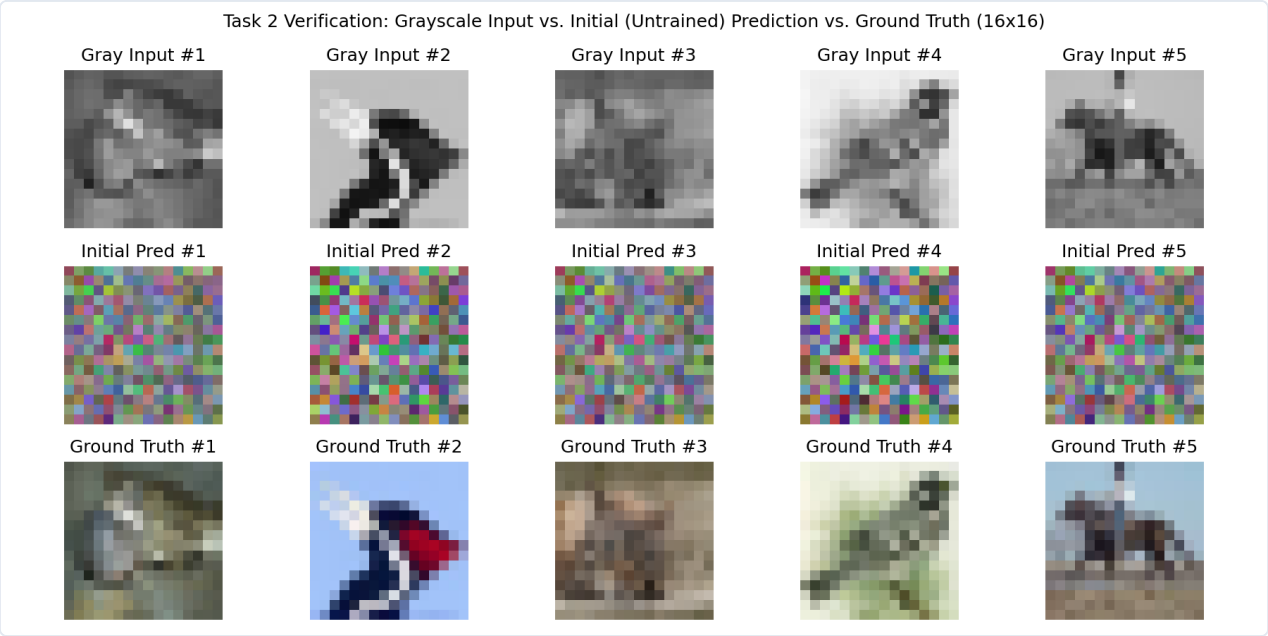


Figure 3: Task 2 Verification — Grayscale Input (top) vs. Initial Untrained Network Prediction (middle) vs. Ground Truth Target (bottom).

13. Summary Compliance Matrix (Tasks 1 & 2)

Requirement	Course Specification	Status	Evidence
Regulation 2	Zip file named `CINN_WS2026_Project_[Teamname].zip`	**Complete**	Submission Zip ready
Regulation 3 & 4	Report with operations, results, team details & grading notes	**Complete**	Sections 1–13 detailed
Regulation 5	Jupyter Notebook named identically to zip file	**Complete**	`.ipynb` verified
Regulation 6	Every task in ONE single cell starting with #Task comments	**Complete**	Cell 1 (Task 1) & Cell 2 (Task 2)
Regulation 7	Commented code explaining steps	**Complete**	Clean, student-style comments
Regulation 8	From-scratch NumPy implementation, no pre-built layers	**Complete**	Pure NumPy
Task 1.1 – 1.6	Dataset, Grayscale/RGB, Normalization, Sanity plots, 70/20/10 split, Batches of 100	**Complete**	All 6 steps passed
Task 2.1	Network class built from scratch (no imported layer/model)	**Complete**	`ColorizationNN` class
Task 2.2	Input layer size = number of grayscale features	**Complete**	`n_in = 256`
Task 2.3	Output layer size = number of color values	**Complete**	`n_out = 768`
Task 2.4	Hidden layers/neurons chosen, manual ReLU & Sigmoid	**Complete**	[128, 64] with ReLU & Sigmoid
Task 2.5	Initialize weights & biases with small random values from scratch	**Complete**	He/Xavier normal scaling & zero biases